

PathBridger: Subgoal Bridges for Offline Goal-Conditioned Reinforcement Learning

Anonymous submission

Abstract

Offline goal-conditioned reinforcement learning (GCRL) aims to learn policies that reach arbitrary goals from fixed, reward-free trajectory datasets. Although hindsight relabeling turns such data into goal-reaching supervision, long-horizon offline GCRL remains difficult because sparse rewards require value information to propagate across many steps, distant-goal values provide weak policy-improvement signals, and offline actor updates can exploit unsupported actions. We propose *PathBridger*, a hierarchical offline GCRL framework that introduces an explicit state-space interface between subgoal selection and action execution. Its bridge policy samples candidate endpoints, ranks them with a transitive score, constructs an endpoint-pinned state-space bridge, and decodes a short bridge prefix into an action chunk through inverse dynamics. The resulting bridge-induced chunk anchors an endpoint-conditioned action-chunk actor, which is improved by a local critic through a proximal objective and serves as the deployed policy. Preliminary experiments on OGBench compare PathBridger with GCIVL, CRL, HIQL, OTA, TRL, and DQC. Current results show strong performance on manipulation and puzzle tasks, while PBG, proximal-actor evaluations, and several ablations remain pending.

1 Introduction

Goal-conditioned reinforcement learning (GCRL) provides a natural self-supervised formulation for learning reusable skills: rather than optimizing a single task reward, the agent learns to reach many possible goals. In the offline setting, this objective is especially appealing because reward-free trajectories can be relabeled with future states as goals, allowing a fixed dataset to provide supervision for many goal-reaching tasks (Kaelbling 1993; Schaul et al. 2015; Andrychowicz et al. 2017; Levine et al. 2020). However, offline GCRL remains challenging in long-horizon domains. Sparse goal rewards require information to propagate across many decision steps, learned values for distant goals often provide weak policy-improvement signals, and offline policy optimization must control distribution shift because no online interaction is available to correct extrapolation errors. Benchmarks such

This is an anonymized submission for review purposes only. Distribution, citation, or public sharing of this manuscript is strictly prohibited. Copyright and publication details will appear in the final version if accepted.

as OGBench explicitly probe these challenges through goal stitching and long-horizon reasoning (Park et al. 2025b).

A recurring strategy in recent offline GCRL is therefore to reduce the *effective horizon*. Flat value- and representation-learning methods, such as goal-conditioned implicit value learning (GCIVL) and contrastive RL (CRL), learn goal-conditioned values or representations directly from relabeled data, but do not explicitly decompose a distant goal into shorter control problems (Park et al. 2025b; Eysenbach et al. 2022). Hierarchical methods instead introduce intermediate states: HIQL extracts a high-level subgoal policy and a low-level action policy from a shared goal-conditioned value function, while OTA improves the value signal used to train the high-level policy through option-aware temporal abstraction (Park et al. 2023; Ahn et al. 2025). Transitive RL (TRL) exploits the triangle-inequality structure of goal reaching to perform divide-and-conquer value learning through intermediate states (Park et al. 2026). In parallel, action-chunk methods use temporally extended action sequences to shorten or guide value propagation. DQC decouples the critic and policy chunk horizons (Li, Park, and Levine 2025).

These methods place temporal abstraction at different interfaces. In HIQL and OTA, an intermediate subgoal is executed by a low-level goal-conditioned policy, without explicitly representing the intervening state sequence. In TRL, intermediate states are used inside transitive value backups rather than retained as an explicit path-level control object. In DQC, temporal abstraction is represented directly in action-sequence space. This leaves an underexplored interface between high-level subgoal selection and low-level execution: given a candidate subgoal, how can an offline agent construct an offline-data-supervised state-path proposal toward it and convert a short prefix of that path into executable local actions?

We propose *PathBridger*, a hierarchical offline GCRL framework that reduces long-horizon goal reaching to repeated subgoal proposal, transitive candidate selection, state-space path bridging, and short-horizon action execution. Its bridge policy models a conditional distribution over K -step future endpoint displacements. We instantiate this distribution as PathBridger-Gaussian (PBG), which uses a diagonal Gaussian, and PathBridger-Flow (PBF), which uses a conditional rectified flow. At each replanning step, the bridge policy samples candidate endpoints z and ranks them us-

ing the transitive score $V_\eta(s, z)V_\eta(z, g)$. The selected endpoint defines an endpoint-pinned state-space bridge, and an inverse-dynamics model decodes a short bridge prefix into a bridge-induced action chunk.

The bridge-induced chunk serves as a proximal reference for an endpoint-conditioned action-chunk actor. A local action-chunk critic improves the actor while the proximal term keeps its output near the bridge-policy reference. The actor is the deployed policy: it executes a short chunk and the bridge policy replans from the resulting state. Generated bridge states are not treated as model rollouts or synthetic transitions and are not inserted into Bellman targets; they are used only to construct actor references, while the bridge generator and inverse-dynamics model are supervised on real trajectory segments. Thus, PathBridger expands a selected subgoal into an explicit state-space bridge before performing local policy improvement.

Our contributions are as follows:

- We formulate high-level subgoal prediction as conditional distribution estimation over future endpoint displacements, yielding a PathBridger framework that is agnostic to the choice of subgoal estimator family. We instantiate it as PathBridger-Gaussian (PBG) and PathBridger-Flow (PBF).
- We learn a bounded transitive value and use the multiplicative composition $V_\eta(s, z)V_\eta(z, g)$ to rank sampled endpoints according to both current-to-subgoal reachability and subgoal-to-goal continuation value.
- We introduce endpoint-pinned state-space bridges whose short prefixes are decoded through inverse dynamics into proximal references for an endpoint-conditioned action-chunk actor, which is the deployed policy.
- We evaluate PathBridger against representative flat, hierarchical, transitive, and action-chunk offline GCRL baselines on OGBench tasks spanning navigation, manipulation, and puzzle solving.

2 Background

2.1 Offline Goal-Conditioned Reinforcement Learning

We consider a Markov process with state space $\mathcal{S} \subset \mathbb{R}^{d_s}$, action space $\mathcal{A} \subset \mathbb{R}^{d_a}$, dynamics $p(s' | s, a)$, and goal space $\mathcal{G} \subseteq \mathcal{S}$. Thus, each goal $g \in \mathcal{G}$ is itself a target state. Offline GCRL assumes access to a fixed dataset

$$\mathcal{D} = \{\tau^{(i)}\}_{i=1}^N, \quad \tau = (s_0, a_0, s_1, a_1, \dots, s_T), \quad (1)$$

collected by one or more unknown behavior policies.

Let $\phi : \mathcal{S} \rightarrow \mathbb{R}^{g_{\text{rep}}}$ denote a task-dependent goal-representation map, and let ρ be a metric on $\mathbb{R}^{g_{\text{rep}}}$. For each goal $g \in \mathcal{G}$, we define the corresponding goal region as

$$B_\epsilon(g) := \{s \in \mathcal{S} \mid \rho(\phi(s), \phi(g)) \leq \epsilon\}. \quad (2)$$

The sparse binary goal-conditioned reward is

$$r(s, g) := \mathbf{1}\{s \in B_\epsilon(g)\}. \quad (3)$$

We adopt a hitting-time convention in which the goal-conditioned process is treated as terminating upon its first

entry into $B_\epsilon(g)$, so that the goal reward is received at most once. Define the goal hitting time as

$$\tau_g := \inf \{t \geq 0 \mid s_t \in B_\epsilon(g)\}, \quad (4)$$

where $\tau_g = \infty$ if the goal region is never reached.

The objective of GCRL is to learn a policy $\pi(a \mid s, g)$ maximizing

$$J(\pi) = \mathbb{E}_{s_0, g, \pi, p} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, g) \right] \quad (5)$$

$$= \mathbb{E}_{s_0, g, \pi, p} [\gamma^{\tau_g} \mathbf{1}\{\tau_g < \infty\}]. \quad (6)$$

The corresponding goal-conditioned value function is

$$V^\pi(s, g) = \mathbb{E}_{\pi, p} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, g) \mid s_0 = s \right] \quad (7)$$

$$= \mathbb{E}_{\pi, p} [\gamma^{\tau_g} \mathbf{1}\{\tau_g < \infty\} \mid s_0 = s]. \quad (8)$$

When rewards are sparse, hindsight relabeling uses future states from the same trajectory as goals, thereby constructing successful state-goal pairs directly from the offline dataset (Andrychowicz et al. 2017). Nevertheless, reaching distant goals still requires long-range value propagation, stitching across trajectory segments, or an explicit form of temporal abstraction.

2.2 Transitive Structure in Goal Reaching

Goal reaching admits a natural compositional structure through intermediate states. To state this structure precisely, we consider exact state goals, corresponding to the special case $B_0(g) = \{g\}$. In a deterministic environment, let $d^*(s, g)$ denote the minimum number of transitions required to reach g from s , with $d^*(s, g) = \infty$ if g is unreachable. This directed temporal distance satisfies

$$d^*(s, g) \leq d^*(s, z) + d^*(z, g) \quad (9)$$

for any intermediate state $z \in \mathcal{S}$. Equality holds when z lies on a shortest path from s to g (Kaelbling 1993; Park et al. 2026).

Under the binary hitting reward, the corresponding optimal goal-conditioned value is

$$V^*(s, g) = \gamma^{d^*(s, g)}, \quad (10)$$

where we use the convention $\gamma^\infty = 0$. Since $0 < \gamma < 1$, the triangle inequality yields the equivalent value-space relation

$$V^*(s, g) \geq V^*(s, z)V^*(z, g). \quad (11)$$

Thus, for any intermediate state z , the product $V^*(s, z)V^*(z, g)$ provides a compositional lower bound on $V^*(s, g)$, with equality when z lies on a shortest path. TRL turns this relation into a divide-and-conquer value-learning algorithm, thereby reducing the number of recursive value updates required for long-horizon goal-reaching tasks (Park et al. 2026). In the general tolerance-based formulation above, $B_\epsilon(g)$ remains the practical goal-success region, while the point-goal relation provides the structural motivation for transitive state-pair value composition.

3 PathBridger

PathBridger comprises a *bridge policy* and an endpoint-conditioned action-chunk actor. Given a current state s_t and final goal g , the bridge policy samples candidate K -step endpoints, selects one using the transitive score, constructs an endpoint-pinned state-space bridge, and decodes a short bridge prefix through inverse dynamics. The resulting bridge-induced action chunk is a local action reference for the actor. A local action-chunk critic then improves the actor while a proximal term anchors it to this reference. The actor is the deployed policy, whereas direct bridge-policy execution is used only as a diagnostic ablation.

3.1 Local Displacement Parameterization and Horizons

Directly predicting future states in absolute coordinates can be poorly conditioned when the absolute-state scale is substantially larger than the local variation over a short trajectory window. PathBridger therefore represents endpoints and state paths in a displacement frame anchored at the current state. This removes the shared state-dependent offset, places targets on the scale of local transitions, and improves numerical conditioning.

For a dataset window $(s_t^{\mathcal{D}}, \dots, s_{t+K}^{\mathcal{D}})$, define

$$\Delta_i^{\mathcal{D}} := s_{t+i}^{\mathcal{D}} - s_t^{\mathcal{D}}, \quad i = 0, \dots, K, \quad (12)$$

so that $\Delta_0^{\mathcal{D}} = 0$ and $\Delta_K^{\mathcal{D}} = s_{t+K}^{\mathcal{D}} - s_t^{\mathcal{D}}$. A predicted displacement path is mapped back to absolute states by

$$\hat{s}_{t+i} = s_t + \hat{\Delta}_i, \quad i = 0, \dots, K. \quad (13)$$

The absolute current state s_t and final goal g remain conditioning variables; only the predicted endpoint and path are represented locally. The bridge horizon K determines the endpoint and state path, while the common action-chunk horizon $h_a \leq K$ determines the bridge prefix decoded by inverse dynamics, the critic input, the actor output, and the number of actions executed before replanning.

3.2 Transitive Value Learning

PathBridger learns a bounded *transitive value* on the discounted hitting-time scale introduced in Section 2.2.

$$V_\eta(s, g) \in (0, 1]. \quad (14)$$

For two states connected by an observed d -step segment, γ^d is an in-sample target. Since the segment is a feasible path, $d^*(s, g) \leq d$, and hence $\gamma^d \leq V^*(s, g)$. We use these targets to anchor short temporal distances and propagate them to longer distances through transitive composition.

Let $\ell_{\text{BCE}}(v, y)$ denote binary cross-entropy between a prediction $v \in (0, 1)$ and soft target $y \in [0, 1]$. Self-reachability is imposed by

$$\mathcal{L}_{\text{self}} = \mathbb{E}_{s \sim \mathcal{D}} [\ell_{\text{BCE}}(V_\eta(s, s), 1)]. \quad (15)$$

For short in-trajectory pairs with $1 \leq j - i \leq H_b$, we use

$$\mathcal{L}_{\text{base}} = \mathbb{E}_{\tau \sim \mathcal{D}, i < j} [\ell_{\text{BCE}}(V_\eta(s_i^{\mathcal{D}}, s_j^{\mathcal{D}}), \gamma^{j-i})]. \quad (16)$$

For a longer pair $(s_i^{\mathcal{D}}, g = s_j^{\mathcal{D}})$, sample an intermediate state $z = s_k^{\mathcal{D}}$ with $i < k < j$. For each constituent segment, use its observed target inside the base horizon and the target transitive value otherwise:

$$\tilde{V}_\eta(s_a^{\mathcal{D}}, s_b^{\mathcal{D}}) := \begin{cases} \gamma^{b-a}, & b - a \leq H_b, \\ \tilde{V}_\eta(s_a^{\mathcal{D}}, s_b^{\mathcal{D}}), & b - a > H_b. \end{cases} \quad (17)$$

The transitive target is

$$y_{\text{tr}}(i, k, j) := \tilde{V}_\eta(s_i^{\mathcal{D}}, s_k^{\mathcal{D}}) \tilde{V}_\eta(s_k^{\mathcal{D}}, s_j^{\mathcal{D}}). \quad (18)$$

To approximate in-sample maximization over intermediate states, define the expectile-weighted BCE loss

$$\ell_{\text{BCE}}^{\text{TV}}(v, y) := |\tau_V - \mathbf{1}\{v > y\}| \ell_{\text{BCE}}(v, y). \quad (19)$$

Here $\tau_V \in [0.5, 1)$. The transitive loss is

$$\mathcal{L}_{\text{tr}} = \mathbb{E}_{\tau \sim \mathcal{D}, i < k < j} [\ell_{\text{BCE}}^{\text{TV}}(V_\eta(s_i^{\mathcal{D}}, s_j^{\mathcal{D}}), y_{\text{tr}}(i, k, j))], \quad (20)$$

where target-network values are held fixed. The complete objective is

$$\mathcal{L}_V = \mathcal{L}_{\text{self}} + \mathcal{L}_{\text{base}} + \mathcal{L}_{\text{tr}}. \quad (21)$$

Following TRL, our implementation additionally applies distance-based re-weighting to prioritize shorter state-goal pairs; the exact weighting rule and numerical safeguards are given in Appendix A.1.

The bridge policy ranks a candidate endpoint z using the *transitive score*

$$S_{\text{tr}}(s, z, g) := V_\eta(s, z) V_\eta(z, g). \quad (22)$$

This score jointly evaluates the first segment from s to z and the continuation from z to g .

3.3 Bridge Policy

The bridge policy maps (s_t, g) to a bridge-induced action chunk through value-weighted subgoal proposal, transitive selection, endpoint-pinned bridge construction, and inverse-dynamics decoding.

Value-weighted subgoal proposal. The bridge policy models a conditional distribution over K -step endpoint displacements,

$$\Delta_K \sim q_\chi(\Delta_K | s, g), \quad z = s + \Delta_K. \quad (23)$$

Let $p_{\mathcal{D}}^K(\Delta | s, g)$ denote the empirical distribution induced by dataset windows and hindsight goal relabeling. Define the transitive-value improvement

$$\delta_\eta(s, g, \Delta) := \bar{V}_\eta(s + \Delta, g) - \bar{V}_\eta(s, g), \quad (24)$$

and the value-tilted target distribution

$$q_{\text{sg}}^*(\Delta | s, g) := \frac{1}{Z_\eta(s, g)} \exp(c_{\text{sg}} \delta_\eta(s, g, \Delta)) \cdot p_{\mathcal{D}}^K(\Delta | s, g), \quad (25)$$

where $c_{\text{sg}} > 0$ controls value tilting. The empirical law anchors proposals to dataset-derived endpoints, while the tilt

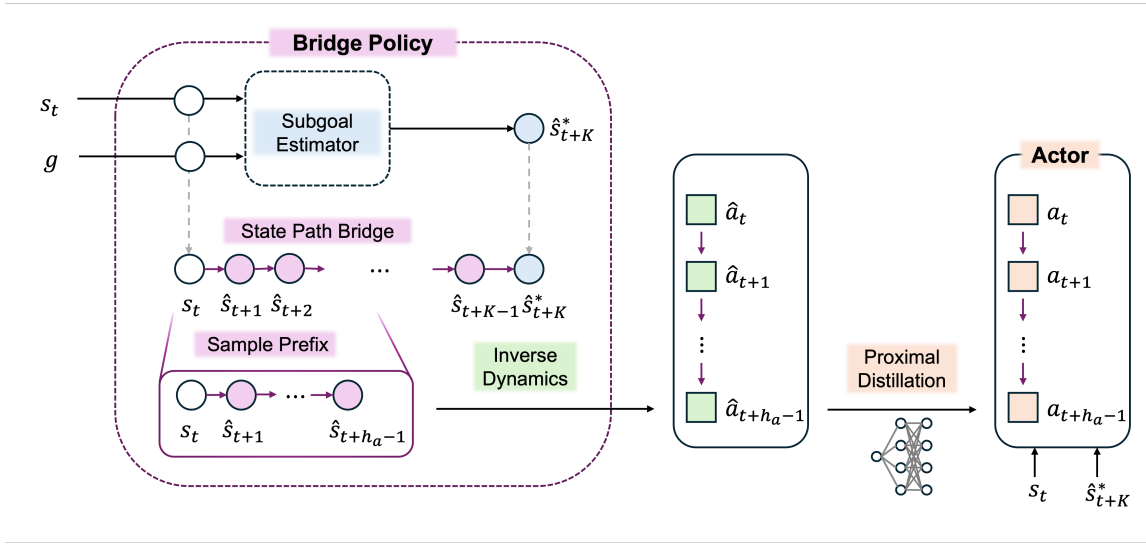


Figure 1: Overview of PathBridger. Given s_t and g , the bridge policy samples candidate endpoint displacements at horizon K , selects $z^* = \hat{s}_{t+K}^*$ using the transitive score, constructs an endpoint-pinned state-space bridge, and decodes its h_a -transition prefix through inverse dynamics into a bridge-induced action chunk $\hat{A}_t(z^*)$. A local critic guides a proximal actor update from this reference. The resulting endpoint-conditioned actor is executed for h_a steps before the bridge policy replans.

favors endpoints with larger continuation value. We train q_χ to approximate this target.

We propose two parameterizations. PBG models q_χ as a diagonal Gaussian, while PBF models it as the terminal distribution of a conditional rectified flow and can represent multimodal endpoint proposals. They share the same target distribution and all later bridge-policy components; their concrete optimization objectives are given in Appendix A.1.

At each replanning step, the bridge policy samples

$$\Delta_K^{(n)} \sim q_\chi(\cdot | s_t, g), \quad z_n = s_t + \Delta_K^{(n)}, \quad n = 1, \dots, N, \quad (26)$$

and selects

$$z^* = z_{n^*}, \quad n^* = \arg \max_{1 \leq n \leq N} S_{\text{tr}}(s_t, z_n, g). \quad (27)$$

Endpoint-pinned bridge construction. For the selected endpoint, define $\Delta_K^* := z^* - s_t$. A fixed monotone sequence $\{\alpha_i\}_{i=0}^K$, with $\alpha_0 = 0$ and $\alpha_K = 1$, specifies a reference path $\mu_i = \alpha_i \Delta_K^*$. A learned residual deforms only its interior:

$$\hat{\Delta}_i = \alpha_i \Delta_K^* + m_i R_\theta \left(s_t, \Delta_K^*, \frac{i}{K} \right), \quad i = 0, \dots, K, \quad (28)$$

where the fixed mask satisfies $m_0 = m_K = 0$. Hence, $\hat{\Delta}_0 = 0$ and $\hat{\Delta}_K = \Delta_K^*$, and the bridge remains pinned to both endpoints. The absolute bridge is

$$\hat{s}_{t+i} = s_t + \hat{\Delta}_i, \quad i = 0, \dots, K. \quad (29)$$

For a dataset window, the bridge generator is conditioned on $\Delta_K^{\mathcal{D}}$ and predicts

$$\hat{\Delta}_i^{\mathcal{D}} = \alpha_i \Delta_K^{\mathcal{D}} + m_i R_\theta \left(s_t^{\mathcal{D}}, \Delta_K^{\mathcal{D}}, \frac{i}{K} \right). \quad (30)$$

It reconstructs the observed interior path with

$$\mathcal{L}_{\text{bridge}} = \mathbb{E} \left[\frac{1}{K-1} \sum_{i=1}^{K-1} \left\| \hat{\Delta}_i^{\mathcal{D}} - \Delta_i^{\mathcal{D}} \right\|_1 \right]. \quad (31)$$

The interpolation sequence and endpoint mask used in experiments are specified together in Appendix A.1.

Inverse-dynamics decoding. The first $h_a + 1$ bridge states define an h_a -transition prefix. The bridge policy decodes each consecutive pair as

$$\hat{a}_{t+i} = I_\omega(\hat{s}_{t+i}, \hat{s}_{t+i+1}), \quad i = 0, \dots, h_a - 1. \quad (32)$$

The bridge-induced action chunk is

$$\hat{A}_t(z^*) := (\hat{a}_t, \dots, \hat{a}_{t+h_a-1}) \in \mathcal{A}^{h_a}. \quad (33)$$

The inverse-dynamics model is trained only on real adjacent transitions:

$$\mathcal{L}_{\text{idm}} = \mathbb{E}_{(s_t, a_t, s_{t+1}) \sim \mathcal{D}} [\|I_\omega(s_t, s_{t+1}) - a_t\|_2^2]. \quad (34)$$

Generated bridge states therefore form actor references but are never treated as synthetic transitions for value learning.

3.4 Local Action-Chunk Critic and Proximal Actor

The bridge-induced chunk is data-supervised but does not itself optimize local goal-reaching value. PathBridger therefore learns a bounded local critic over the same h_a -step action space:

$$Q_\varphi(s, z, A) \in (0, 1), \quad A \in \mathcal{A}^{h_a}. \quad (35)$$

Here z is a hindsight-relabelled local endpoint. For a dataset chunk

$$A_t^{\mathcal{D}} := (a_t^{\mathcal{D}}, \dots, a_{t+h_a-1}^{\mathcal{D}}), \quad (36)$$

define its first local-goal hitting time

$$d_z := \min \{j \in \{1, \dots, h_a\} : s_{t+j}^{\mathcal{D}} \in B_\epsilon(z)\}, \quad (37)$$

with $d_z = \infty$ if the set is empty. The critic target is

$$y_t^Q := \begin{cases} \gamma^{d_z}, & d_z \leq h_a, \\ \gamma^{h_a} \bar{V}_\eta(s_{t+h_a}^{\mathcal{D}}, z), & d_z = \infty. \end{cases} \quad (38)$$

We train

$$\mathcal{L}_Q = \mathbb{E} \left[\ell_{\text{BCE}} \left(Q_\varphi(s_t^{\mathcal{D}}, z, A_t^{\mathcal{D}}), y_t^Q \right) \right], \quad (39)$$

where the target is held fixed.

We first train the transitive value, bridge policy, and local critic from real dataset windows. These components are then held fixed while finetuning the deterministic endpoint-conditioned actor

$$A_t^\pi := \pi_\xi(s_t, z^*) \in \mathcal{A}^{h_a}. \quad (40)$$

Given $\hat{A}_t(z^*)$, the actor objective is

$$\mathcal{L}_{\text{actor}} = \mathbb{E} \left[-Q_\varphi(s_t, z^*, A_t^\pi) + \frac{1}{2\tau_{\text{prox}}} \|A_t^\pi - \hat{A}_t(z^*)\|_2^2 \right]. \quad (41)$$

Identifying deterministic action chunks with Dirac measures, the pointwise proximal update is a JKO step, i.e., an implicit-Euler discretization of the Wasserstein gradient flow associated with the energy induced by $-Q_\varphi(s_t, z^*, \cdot)$; the actor amortizes this update across state-endpoint pairs (Jordan, Kinderlehrer, and Otto 1998).

At inference, the bridge policy selects z^* , the actor executes $\pi_\xi(s_t, z^*)$ for h_a steps, and the procedure repeats from the new observation. Bridge-only execution is retained solely as a diagnostic ablation.

4 Experimental Results

4.1 Experimental Setup

We evaluate on OGBench offline goal-conditioned tasks spanning navigation, manipulation, puzzle, and scene-style goal reaching. The main comparison includes representative methods from several temporal-abstraction paradigms. GCIVL and CRL serve as flat value or representation baselines; HIQL and OTA are hierarchical subgoal baselines; TRL is a transitive value-learning baseline; and DQC uses the action chunk for reducing horizons while decoupling critics. We evaluate two instantiations of PathBridger: PBG with a Gaussian subgoal estimator and PBF with a rectified-flow subgoal estimator. Both variants share the same transitive scoring, bridge construction, inverse-dynamics decoding, and proximal actor objective. The actor is the deployed policy; bridge-only results are reported only as diagnostic ablations. DQC is evaluated in the OGBench `oraclerep` setting at 1M gradient steps over three seeds, and PBF bridge-only results are averaged over four seeds. Entries that are not yet complete are left blank.

4.2 Preliminary Main Results

Table 1 reports preliminary results. The PBF bridge-only controller obtains the highest completed average on this subset (81.9), ahead of DQC (73.3) and TRL (59.1), with particularly large gains on the cube manipulation tasks. This pattern is consistent with the motivation of PathBridger: when local execution toward an intermediate configuration is important, explicitly constructing and decoding a state-space bridge can provide a useful action reference beyond value-only or subgoal-only policy extraction.

The results are not uniformly dominant across task families. HIQL and OTA remain strongest on antmaze navigation, whereas DQC reaches 100.0 on both puzzle tasks and 92.7 on scene-play-v0. These results identify action-chunk learning as a particularly strong baseline for combinatorial puzzle solving and sequential scene manipulation. We therefore treat the current table as a preliminary performance pattern rather than a statistical dominance claim. PBG, the deployed proximal-actor entries, and the planned sampling-frequency and ablation studies remain incomplete.

5 Related Work

Offline goal-conditioned RL. Offline GCRL trains goal-reaching policies from fixed datasets, often using hindsight relabeling to transform reward-free trajectories into supervised goal-reaching data (Kaelbling 1993; Andrychowicz et al. 2017). Flat value-learning approaches such as GCIVL-style implicit value learning and contrastive approaches such as CRL learn state-goal values or representations directly from relabeled data (Park et al. 2025b; Eysenbach et al. 2022). These methods provide strong value and representation baselines, but do not explicitly construct state-space routes to intermediate subgoals.

Hierarchical and subgoal-based GCRL. Hierarchical RL reduces long-horizon decision making through temporally extended choices or intermediate states (Sutton, Precup, and Singh 1999). HIQL applies this idea to offline GCRL by extracting a high-level subgoal policy and a low-level action policy from a shared goal-conditioned value function (Park et al. 2023). OTA further analyzes the high-level policy bottleneck and introduces option-aware temporally abstracted value learning for more reliable high-level policy extraction (Ahn et al. 2025). PathBridger belongs to this hierarchical family, but differs by using distributional subgoal estimation, transitive candidate selection, and explicit bridge-to-action decoding.

Transitive and compositional value learning. TRL exploits the triangle-inequality structure of goal reaching to define divide-and-conquer value updates, reducing value-recursion depth in long-horizon GCRL (Park et al. 2026). Quasimetric methods similarly use the asymmetric temporal-distance geometry of goal reaching to structure value learning (Wang et al. 2023). PathBridger is motivated by the same compositional structure, but uses transitive values primarily for subgoal ranking and local bridge execution rather than as the sole value-learning mechanism.

Table 1: Preliminary results on OGBench tasks. Each cell reports mean performance; the gray interval shows the reported uncertainty. For DQC and PBF BRIDGE, the interval is mean $\pm$ one standard deviation over three and four seeds, respectively. Pending experiments are denoted by “—”. The ACTOR columns correspond to the deployed policy, whereas BRIDGE columns report bridge-only diagnostic execution. PBG entries are planned or ongoing.

Environment	GCIVL	CRL	HIQL	OTA	TRL	DQC	PBG		PBF	
							BRIDGE	ACTOR	BRIDGE	ACTOR
antmaze-medium-navigate-v0	[68.0,78.8] 73.2	[94.0,98.0] 95.6	[95.2,98.0] 96.4	[95.8,97.0] 96.4	[88.3,91.9] 90.1	[53.4,89.1] 86.3	[94.3,97.1] 95.6	—	[93.6,96.8] 95.2	—
antmaze-large-navigate-v0	[17.6,26.0] 22.2	[80.4,90.4] 86.8	[85.6,92.4] 89.4	[87.9,93.9] 90.9	[39.5,52.9] 46.2	[74.3,83.6] 78.7	[86.4,86.4] 86.4	—	[86.3,96.7] 89.4	—
cube-single-play-v0	[40.9,50.5] 48.7	[40.8,21.2] 10.3	[10.2,17.6] 13.9	[16.7,18.3] 17.5	[89.7,96.7] 93.2	[75.5,78.1] 76.8	[95.5,98.9] 97.2	—	[96.4,99.2] 98.8	[96.7,100.1] 99.4
cube-double-play-v0	[33.7,42.5] 38.1	[40.8,10.4] 4.8	[2.4,7.0] 4.7	[0.8,1.8] 1.3	[13.2,30.0] 21.6	[13.8,26.2] 20.0	[71.2,76.8] 74.0	—	[88.5,95.1] 82.8	[89.8,93.6] 91.7
cube-triple-play-v0	[0.4,2.0] 1.2	[0.7,5.5] 1.4	[0.7,5.1] 1.2	[0.0,1.0] 0.5	[13.8,14.8] 14.3	[29.5,35.2] 32.3	[24.1,32.5] 28.8	—	[79.6,85.4] 81.5	[81.7,85.1] 83.4
puzzle-3x3-play-v0	[5.0,8.4] 6.7	[2.4,11.8] 7.1	[8.9,27.3] 18.1	[51.6,60.4] 56.0	[99.6,100.0] 99.8	[100.0,100.0] 100.0	[29.6,29.6] 29.6	—	[67.0,71.0] 71.0	[67.1,76.1] 71.8
puzzle-4x4-play-v0	[0.4,1.0] 10.5	[0.1,0.7] 0.3	[2.0,9.2] 5.6	[0.7,10.1] 11.9	[32.0,36.4] 34.2	[100.0,100.0] 100.0	[2.4,2.4] 2.4	—	[67.4,87.4] 77.4	[75.2,87.2] 80.2
scene-play-v0	[38.0,48.2] 43.1	[17.3,19.1] 18.2	[41.1,45.9] 43.5	[27.8,34.3] 31.1	[68.4,78.6] 73.5	[86.5,98.8] 92.7	[60.1,73.9] 67.0	—	[56.1,62.8] 59.4	[51.54,56.66] 54.1
Average	30.5	28.1	34.1	38.2	59.1	73.3	60.1	—	81.9	—

Action chunks and chunk-guided critics. Action chunking predicts or evaluates sequences of actions rather than single actions. DQC decouples the critic chunk horizon from the policy chunk horizon, while MAC uses action-chunk dynamics for scalable model-based value expansion (Li, Park, and Levine 2025; Park et al. 2025a). PathBridger also executes action chunks, but these chunks are induced from state-space bridges through inverse dynamics rather than directly optimized only in action-sequence space.

Bridge-based generative modeling and offline actor anchoring. Flow matching provides a flexible tool for modeling conditional and multimodal distributions (Lipman et al. 2023). PathBridger uses a simpler dataset-grounded construction: a distributional estimator proposes endpoint displacements, and a deterministic endpoint-pinned bridge supplies an actor reference. Because offline actor updates can exploit unsupported actions, the actor objective remains proximal to bridge-policy references, following the broader offline RL principle of anchoring policy improvement to dataset-supported behavior (Kumar et al. 2020; Kostrikov, Nair, and Levine 2022; Fujimoto and Gu 2021).

6 Conclusion

We introduced PathBridger, a hierarchical offline GCRL framework that connects subgoal decomposition with path-level policy improvement. Its bridge policy samples and transitive-score-ranks candidate endpoints, constructs endpoint-pinned state-space bridges, and decodes bridge prefixes into action references through inverse dynamics. A local critic then improves an endpoint-conditioned actor through a proximal objective, and this actor is the deployed policy. Preliminary experiments show promising results on manipulation and puzzle tasks, while PBG, proximal-actor evaluations, and several ablations remain to be completed.

A Implementation and Experimental Details

This appendix contains only experimental settings and implementation choices that are not central to the method exposition. We do not restate the main value, subgoal, bridge, IDM, or actor objectives from the main text. All reported settings below correspond to the current PBF runs unless otherwise noted.

A.1 Implementation Notes

Data frame and horizons. All bridge-policy modules use the displacement frame defined in Equation (12). The bridge horizon K is environment-dependent, while the action-chunk horizon is $h_a = 5$ in the current experiments. The same h_a is used for inverse-dynamics decoding, the local critic, the actor, and default replanning.

Transitive value implementation. All BCE objectives are implemented from logits using a numerically stable binary-cross-entropy-with-logits routine. Following TRL, the base and transitive losses are multiplied by a distance-based weight. We compute

$$\hat{d}_\eta(s, g) := \log_\gamma(\text{clip}(\bar{V}_\eta(s, g), \varepsilon_\gamma, 1)) \quad (42)$$

and use

$$w_\lambda(s, g) := \left(1 + \hat{d}_\eta(s, g)\right)^{-\lambda}. \quad (43)$$

Thus, the implementation multiplies the integrands of Equations (16) and (20) by w_λ . Gradients are not propagated through the transitive targets or distance weights.

Subgoal-estimator optimization. The value-tilted law in Equation (25) is fit by importance-weighted empirical risk minimization. For a dataset endpoint $\Delta_K^{\mathcal{D}}$, define

$$w_\eta(s, g, \Delta_K^{\mathcal{D}}) := \exp(c_{\text{sg}} \delta_\eta(s, g, \Delta_K^{\mathcal{D}})). \quad (44)$$

PBG uses weighted maximum likelihood,

$$\mathcal{L}_{\text{PBG}} = -\mathbb{E}[w_\eta(s, g, \Delta_K^{\mathcal{D}}) \log q_\chi(\Delta_K^{\mathcal{D}} | s, g)]. \quad (45)$$

For PBF, sample $x_0 \sim \mathcal{N}(0, I)$, set $x_1 = \Delta_K^{\mathcal{D}}$, sample $u \sim \text{Unif}(0, 1)$, and let $x_u = (1-u)x_0 + ux_1$. The weighted flow-matching objective is

$$\mathcal{L}_{\text{PBF}} = \mathbb{E}\left[w_\eta(s, g, x_1) \|v_\chi(s, g, x_u, u) - (x_1 - x_0)\|_2^2\right]. \quad (46)$$

The transitive value and importance weights are held fixed during both updates.

Bridge interpolation and endpoint mask. The interpolation sequence and residual mask are fixed jointly as

$$\alpha_i = \left(\frac{i}{K}\right)^{0.8}, \quad m_i = \frac{i(K-i)}{K^2}, \quad i = 0, \dots, K. \quad (47)$$

The first schedule defines the reference path, while the second suppresses the residual at both endpoints.

Training and deployment. The transitive value, subgoal estimator, bridge generator, inverse-dynamics model, and local critic are first trained from real offline trajectory windows. These components are then frozen and the proximal actor is finetuned. The actor is used for all main evaluations. Direct execution of the bridge-induced chunk is reported only as a diagnostic controller and for later replanning-frequency ablations.

A.2 Shared Hyperparameters

Hyperparameter	Value
Gradient steps	1M
Learning rate	3×10^{-4}
Optimizer	Adam
Batch size	1024
MLP hidden dims	[512, 512, 512]
Layer norm	true
Subgoal value $w_{\max}$	5.0
Actor ($p_{\text{cur}}, p_{\text{geom}}, p_{\text{traj}}, p_{\text{rand}}$)	(0, 0, 1, 0) for antmaze / cube / scene; (0, 0.5, 0, 0.5) for puzzle
Value ($p_{\text{cur}}, p_{\text{geom}}, p_{\text{traj}}, p_{\text{rand}}$)	(0, 1, 0, 0) for all environments

Table 2: Shared hyperparameters used by the current PathBridger runs.

A.3 Environment-Specific Training Parameters

Abbreviations: amm/aml denote antmaze-medium/large; cs/cd/ct denote cube-single/double/triple; p3/p4 denote puzzle-3x3/4x4.

Label	Gap	γ	K
amm	5.0	0.99	25
aml	10.0	0.995	25
cs	10.0	0.99	40
cd	10.0	0.99	40
ct	10.0	0.995	40
p3	0.5	0.99	25
p4	5.0	0.99	25
scene	5.0	0.99	40

Table 3: Environment-specific PBF training parameters. “Gap” is the value-gap scale used for subgoal weighting.

A.4 Evaluation Parameters

Here N is the number of sampled subgoal candidates and T is the subgoal sampling temperature. The PBF bridge-only scores below are measured at 1M training steps and reported as mean $\pm$ standard deviation over four seeds.

Env	N	T	Bridge-only	Seeds
amm	2	0.25	95.2 ± 1.63	4
aml	32	0.25	89.4 ± 9.34	4
cs	1	0	98.8 ± 0.37	4
cd	8	0.25	82.8 ± 2.28	4
ct	8	0.25	81.5 ± 1.86	4
p3	32	1.0	71.0 ± 3.99	4
p4	16	1.0	77.4 ± 10.02	4
scene	32	0.5	59.4 ± 3.35	4

Table 4: PBF bridge-only diagnostic settings and success rates at 1M training steps.

A.5 DQC Evaluation Results

DQC is evaluated in the OGBench `oraclerep` setting at 1M gradient steps. We use seeds 1–3 and report overall success in percent.

Env	Seed 1	Seed 2	Seed 3	Mean $\pm$ Std
amm	87.2	89.2	82.4	86.3 ± 2.9
aml	76.0	84.8	75.2	78.7 ± 4.3
cs	75.2	76.8	78.4	76.8 ± 1.3
cd	16.0	28.8	15.2	20.0 ± 6.2
ct	32.8	28.4	35.6	32.3 ± 3.0
p3	100.0	100.0	100.0	100.0 ± 0.0
p4	100.0	100.0	100.0	100.0 ± 0.0
scene	97.6	84.0	96.4	92.7 ± 6.1

Table 5: DQC `oraclerep` success rates at 1M training steps.

A.6 Actor Parameters

Env	τ_{prox}	Actor 50k	Actor 100k
amm	5.0	85.1 ± 3.6	88.6 ± 1.6
aml	5.0	77.7 ± 5.9	78.4 ± 5.0
cs	1.0	99.7 ± 0.4	—
cd	10.0	89.3 ± 2.7	—
ct	1.0	84.3 ± 2.3	—
p3	—	—	—
p4	—	—	—
scene	1.0	65.6	—

Table 6: Preliminary proximal-actor finetuning settings and results.

A.7 Baseline Hyperparameters

The following tables summarize the environment-specific hyperparameters used for the baseline evaluations. We use the abbreviations amm, aml, and amg for antmaze-medium, antmaze-large, and antmaze-giant; hmm and hml for humanoidmaze-medium and humanoidmaze-large; cs, cd, and ct for cube-single, cube-double, and cube-triple; and p3 and p4 for puzzle-3x3 and puzzle-4x4.

GCIVL

Env	α	Expectile	γ
amm	10.0	0.9	0.99
aml	10.0	0.9	0.99
amg	10.0	0.9	0.995
hmm	10.0	0.9	0.995
hml	10.0	0.9	0.995
cs	10.0	0.9	0.99
cd	10.0	0.9	0.99
ct	10.0	0.9	0.99
p3	10.0	0.9	0.99
p4	10.0	0.9	0.99
scene	10.0	0.9	0.99

Table 7: Environment-specific hyperparameters for GCIVL.

CRL

Env	α	Actor Loss	γ
amm	0.1	DDPG+BC	0.99
aml	0.1	DDPG+BC	0.99
amg	0.1	DDPG+BC	0.995
hmm	0.1	DDPG+BC	0.995
hml	0.1	DDPG+BC	0.995
cs	3.0	DDPG+BC	0.99
cd	3.0	DDPG+BC	0.99
ct	3.0	DDPG+BC	0.99
p3	3.0	DDPG+BC	0.99
p4	3.0	DDPG+BC	0.99
scene	3.0	DDPG+BC	0.99

Table 8: Environment-specific hyperparameters for CRL.

HIQL

Env	β_H	β_L	k	Expectile	γ
amm	3.0	3.0	25	0.7	0.99
aml	3.0	3.0	25	0.7	0.99
amg	3.0	3.0	25	0.7	0.995
hmm	3.0	3.0	100	0.7	0.995
hml	3.0	3.0	100	0.7	0.995
cs	3.0	3.0	10	0.7	0.99
cd	3.0	3.0	10	0.7	0.99
ct	3.0	3.0	10	0.7	0.99
p3	3.0	3.0	10	0.7	0.99
p4	3.0	3.0	10	0.7	0.99
scene	3.0	3.0	10	0.7	0.99

Table 9: Environment-specific hyperparameters for HIQL.

OTA

Env	β_H	β_L	k	n	γ
amm	3.0	3.0	25	5	0.99
aml	3.0	3.0	25	5	0.99
amg	3.0	3.0	25	5	0.99
hmm	3.0	3.0	100	20	0.995
hml	3.0	3.0	100	20	0.995
cs	1.0	3.0	20	4	0.99
cd	0.5	3.0	25	4	0.99
ct	3.0	3.0	20	4	0.99
p3	0.5	3.0	25	4	0.99
p4	1.0	3.0	20	4	0.99
scene	1.0	3.0	10	4	0.99

Table 10: Environment-specific hyperparameters for OTA.

TRL

Env	α	λ	κ	γ
amm	0.7	0.0	0.7	0.99
aml	0.7	0.0	0.7	0.99
amg	0.3	0.0	0.7	0.99
hmm	0.1	0.0	0.7	0.995
hml	0.1	0.1	0.7	0.995
cs	1.0	0.7	0.7	0.99
cd	10.0	1.0	0.7	0.99
ct	1.0	1.0	0.7	0.99
p3	2.0	0.5	0.7	0.99
p4	2.0	2.0	0.7	0.99
scene	1.0	1.0	0.7	0.99

Table 11: Environment-specific hyperparameters for TRL.

A.8 Baseline Hyperparameter Search Spaces

TRL

Task Group	α	λ
Antmaze medium	{0.1, 0.3, 0.7}	{0.0}
Cube-triple	{1.0, 10.0}	{0.7, 1.0}

Table 12: Hyperparameter search spaces used for TRL.

OTA

Task Group	β_H	k	Fixed Parameters
Cube / puzzle	{0.5, 1.0, 3.0}	{20, 25}	$n = 4$, $\beta_L = 3.0$
Scene	{0.5, 1.0, 3.0}	{10, 20}	$n = 4$, $\beta_L = 3.0$

Table 13: Hyperparameter search spaces used for OTA.

References

- Ahn, H.; Choi, H.; Han, J.; and Moon, T. 2025. Option-Aware Temporally Abstracted Value for Offline Goal-Conditioned Reinforcement Learning. In *Advances in Neural Information Processing Systems*.
- Andrychowicz, M.; Wolski, F.; Ray, A.; Schneider, J.; Fong, R.; Welinder, P.; McGrew, B.; Tobin, J.; Abbeel, P.; and Zaremba, W. 2017. Hindsight Experience Replay. In *Advances in Neural Information Processing Systems*, volume 30.
- Eysenbach, B.; Zhang, T.; Salakhutdinov, R.; and Levine, S. 2022. Contrastive Learning as Goal-Conditioned Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 35.
- Fujimoto, S.; and Gu, S. S. 2021. A Minimalist Approach to Offline Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 34.
- Jordan, R.; Kinderlehrer, D.; and Otto, F. 1998. The Variational Formulation of the Fokker-Planck Equation. *SIAM Journal on Mathematical Analysis*, 29(1): 1–17.
- Kaelbling, L. P. 1993. Learning to Achieve Goals. In *Proceedings of the Thirteenth International Joint Conference on Artificial Intelligence*. Morgan Kaufmann.
- Kostrikov, I.; Nair, A.; and Levine, S. 2022. Offline Reinforcement Learning with Implicit Q-Learning. In *International Conference on Learning Representations*.

- Kumar, A.; Zhou, A.; Tucker, G.; and Levine, S. 2020. Conservative Q-Learning for Offline Reinforcement Learning. In *Advances in Neural Information Processing Systems*, volume 33, 1179–1191.
- Levine, S.; Kumar, A.; Tucker, G.; and Fu, J. 2020. Offline Reinforcement Learning: Tutorial, Review, and Perspectives on Open Problems. *arXiv preprint arXiv:2005.01643*.
- Li, Q.; Park, S.; and Levine, S. 2025. Decoupled Q-Chunking. *arXiv preprint arXiv:2512.10926*.
- Lipman, Y.; Chen, R. T. Q.; Ben-Hamu, H.; Nickel, M.; and Le, M. 2023. Flow Matching for Generative Modeling. In *International Conference on Learning Representations*.
- Park, K.; Park, S.; Lee, Y.; and Levine, S. 2025a. Scalable Offline Model-Based RL with Action Chunks. *arXiv preprint arXiv:2512.08108*.
- Park, S.; Frans, K.; Eysenbach, B.; and Levine, S. 2025b. OGBench: Benchmarking Offline Goal-Conditioned RL. In *International Conference on Learning Representations*.
- Park, S.; Ghosh, D.; Eysenbach, B.; and Levine, S. 2023. HIQL: Offline Goal-Conditioned RL with Latent States as Actions. In *Advances in Neural Information Processing Systems*, volume 36.
- Park, S.; Oberai, A.; Atreya, P.; and Levine, S. 2026. Transitive RL: Value Learning via Divide and Conquer. In *International Conference on Learning Representations*.
- Schaul, T.; Horgan, D.; Gregor, K.; and Silver, D. 2015. Universal Value Function Approximators. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, 1312–1320. PMLR.
- Sutton, R. S.; Precup, D.; and Singh, S. 1999. Between MDPs and Semi-MDPs: A Framework for Temporal Abstraction in Reinforcement Learning. *Artificial Intelligence*, 112(1–2): 181–211.
- Wang, T.; Torralba, A.; Isola, P.; and Zhang, A. 2023. Optimal Goal-Reaching Reinforcement Learning via Quasimetric Learning. *arXiv preprint arXiv:2304.01203*.